Distributed Systems

Opty Report
Team M3 #1.01

Autumn 2026

Code: CPDS

Inderjot Singh (inderjot.singh@estudiantat.upc.edu)

Fawzi Daoud (fawzi.daoud@estudiantat.upc.edu)

Ida Kallevik Tollaksen(ida.kallevik.tollaksen@estudiantat.upc.edu)

UPC

1 Introduction

In this seminar we implemented Opty, a transaction server in Erlang that uses optimistic concurrency control. Instead of locking data, transactions run freely and are only checked for conflicts when they try to commit, so a transaction is aborted if another one changed the data it used. We studied how the number of clients, the size of the store and the mix of reads and writes affect how many transactions succeed. We also ran the server and the clients on separate Erlang nodes, and compared the original backward validation with forward validation.

2 Experiments and Questions

For experiments 2.1–2.6, each configuration was first ran manually in the Erlang shell. To check how much the results vary between runs, each configuration was then run 10 times using a Python script (run_experiments.py). The script starts a fresh Erlang node for every run, calls opty:start/6 with the same parameters as the manual runs (5 seconds per run). It records the number of transactions and committed transactions for each client. The tables marked as averages show the mean over these 10 runs.

2.1 Number of concurrent clients

Experiment. *Provide figures with experimental results comparing the number of transactions and the success rate (i.e. percentage of committed transactions with respect to the total) when executing with different number of concurrent clients in the system.*

Question. *Discuss the impact of the number of clients on the success rate, and indicate whether the success rate is the same for the different clients.*

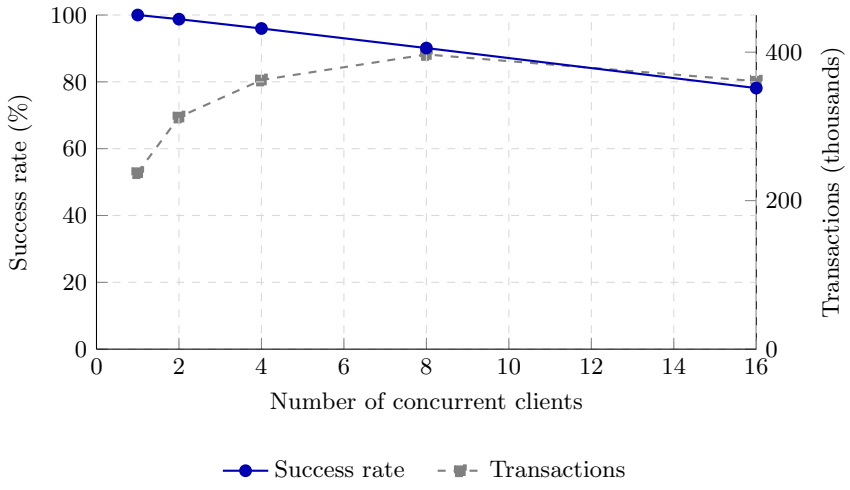


Figure 1: Success rate and number of transactions vs. number of concurrent clients (average of 10 runs).

Clients	Transactions	Committed	Success rate (%)
1	251 824	251 824	100.00
2	322 519	318 714	98.82
4	338 370	324 611	95.93
8	410 912	371 004	90.29
16	381 644	298 685	78.26

Table 1: Manual runs with 1000 entries, 5 reads, 5 writes, 5s.

Clients	Transactions	Committed	Success rate (%)
1	237 681	237 681	100.00
2	312 578	308 697	98.76
4	362 475	347 860	95.97
8	396 886	357 614	90.11
16	360 887	281 996	78.13

Table 2: Average of 10 runs 1000 entries, 5 reads, 5 writes, 5s.

Tables 1 and 2 show the same trend for the manual runs and for the average over 10 runs. The success rate decreases as the number of concurrent clients increases, from 100% with a single client to about 78% with 16 clients. The two tables agree within 0.2 percentage points, which means that a single run already gives an accurate rate.

The decrease in success is caused by conflicts between transactions. A transaction aborts if another client has committed a write to one of the entries it read before the transaction is validated. With more clients, more transactions commit writes at the same time, so the probability of a conflict increases.

The total number of transactions increases with the number of clients up to 8 clients because more clients execute transactions in parallel. With 16 clients the total number of transactions decreases. This is because this experiments was done on a machine with 10 cores, so with 16 clients the processes compete for CPU time. This means a larger share of the work is spent on transactions that are later aborted. The number of transactions per client decreases in all cases, since the clients share the same server, validator and CPU resources.

The success rate was almost the same for all clients. In each run, the difference between the client with the highest and the lowest success rate was at most 0.05 percentage points with 2 clients, 0.3 with 4, 0.7 with 8 and 1.5 with 16 clients. The difference grows a little with more clients, since each client completes fewer transactions and the results become more affected by randomness, but it stays in a small range.

2.2 Number of entries

Experiment. *Provide figures with experimental results comparing the number of transactions and the success rate when executing with different number of entries in the store.*

Question. *Discuss the impact of the number of entries on the success rate, and indicate whether the success rate is the same for the different clients.*

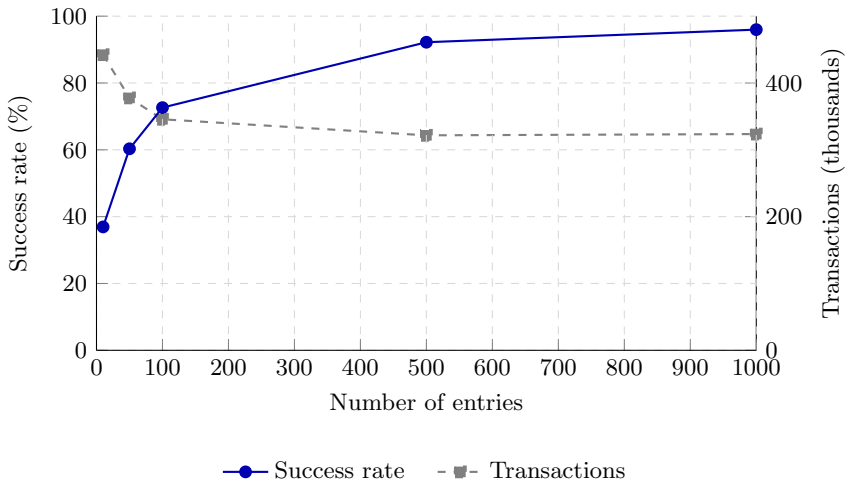


Figure 2: Success rate and number of transactions vs. number of entries (average of 10 runs).

Entries	Transactions	Committed	Success rate (%)
10	457 887	168 751	36.85
50	401 704	242 117	60.27
100	387 876	282 021	72.71
500	367 623	339 277	92.29
1000	362 649	347 939	95.94

Table 3: Manual runs with 10, 50 100, 500 and 1000 entries.

Entries	Transactions	Committed	Success rate (%)
10	442 398	163 388	36.93
50	377 742	227 827	60.31
100	345 728	251 108	72.63
500	321 718	296 615	92.20
1000	323 566	310 511	95.97

Table 4: Average of 10 run with 10, 50 100, 500 and 1000 entries.

Tables 3 and 4 show that the number of entries has a large impact on the success rate. With 10 entries only about 37% of the transactions commit, while with 1000 entries about 96% commit. With few entries, the probability that two concurrent transactions access the same entry is high. A transaction aborts if any of the entries it read has been written by another committed transaction, so a small store gives many conflicts. When the store grows, the reads and writes are spread over more entries and conflicts become fewer as we can see.

The total number of transactions is highest with 10 entries and decreases as the store grows. One reason is that aborted transactions are cheaper because the validator stops at the first entry that replies abort and does not perform any writes. It is also because with only 10 entries, a read often finds the entry

in the local write set of the handler, so it does not need to contact the entry process.

The success rate was similar for all clients. In each run, the per-client rates differed by at most 0.8 percentage points with 10 entries, and at most 0.2 percentage points with 1000 entries. This is because all clients use the same parameters and choose entries uniformly at random.

2.3 Number of read operations

Experiment. *Provide figures with experimental results comparing the number of transactions and the success rate when executing with different number of read operations per transaction.*

Question. *Discuss the impact of the number of read operations on the success rate, and indicate whether the success rate is the same for the different clients.*

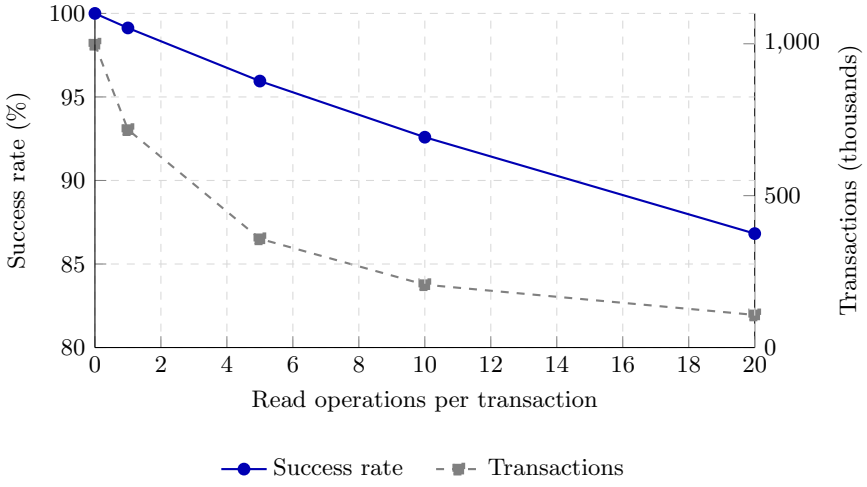


Figure 3: Success rate and number of transactions vs. number of read operations per transaction (average of 10 runs). The success-rate axis starts at 80%.

Reads	Transactions	Committed	Success rate (%)
0	1 145 135	1 145 135	100.00
1	736 116	729 848	99.15
5	362 549	348 137	96.02
10	227 473	210 525	92.55
20	128 891	111 768	86.72

Table 5: Manual runs with 0, 1, 5, 10 and 20 read operations.

Tables 5 and 6 show that the success rate decreases as the number of reads per transaction increases, from 100% with 0 reads to about 87% with 20 reads. With 0 reads the read set is empty, so there is nothing to validate and every transaction commits. This is true even though the transactions still write, because backward validation only checks the read set. With more reads, there

Reads	Transactions	Committed	Success rate (%)
0	998 559	998 559	100.00
1	717 354	711 110	99.13
5	358 017	343 500	95.95
10	206 914	191 583	92.59
20	107 425	93 267	86.82

Table 6: Average of 10 runs with 0, 1, 5, 10 and 20 read operations.

are more entries that another transaction can modify before the validation. Each read also makes the transaction last longer, which gives other clients more time to commit conflicting writes.

The number of transactions decreases a lot with more reads, from about 1.1 million with 0 reads to about 110 000–130 000 with 20 reads. This is because reads are synchronous, meaning the client waits for the value from the entry (through the handler) before it continues. Writes are only stored locally in the handler and do not need a reply.

The success rate was similar for all clients. In each run the per-client rates differed by at most 0.7 percentage points (with 20 reads). With 0 reads all clients committed every transaction.

2.4 Number of write operations

Experiment. *Provide figures with experimental results comparing the number of transactions and the success rate when executing with different number of write operations per transaction.*

Question. *Discuss the impact of the number of write operations on the success rate, and indicate whether the success rate is the same for the different clients.*

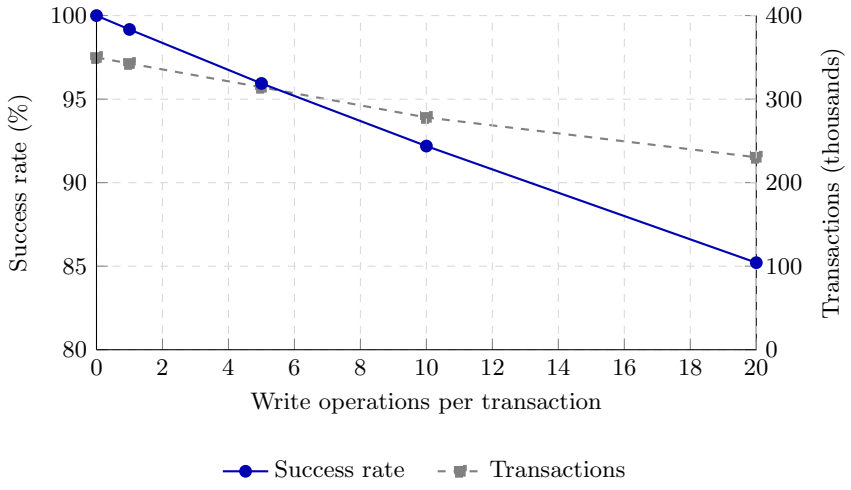


Figure 4: Success rate and number of transactions vs. number of write operations per transaction (average of 10 runs). The success-rate axis starts at 80%.

Writes	Transactions	Committed	Success rate (%)
0	398 463	398 463	100.00
1	396 068	392 794	99.17
5	367 669	352 821	95.96
10	323 647	298 356	92.19
20	275 520	235 026	85.30

Table 7: Manual runs with 0, 1, 5, 10 and 20 write operations.

Table 8: Average of 10 runs: writes per transaction (5 reads).

Writes	Transactions	Committed	Success rate (%)
0	349 832	349 832	100.00
1	342 734	339 892	99.17
5	314 194	301 451	95.94
10	278 145	256 425	92.19
20	230 437	196 344	85.21

Table 9: Average of 10 runs with 0, 1, 5, 10 and 20 write operations.

Tables 7 and 8 show that the success rate decreases as the number of writes per transaction increases, from 100% with 0 writes to about 85% with 20 writes. With 0 writes, no transaction ever changes an entry, so the timestamps never change and all validations succeed. With more writes, each committed transaction updates more entries. This invalidates the reads of other transactions that are running at the same time. The writes of a transaction do not make that transaction itself more likely to abort. They make the other transactions more likely to abort.

The number of transactions decreases with more writes, but much less than with more reads (from about 400 000 to about 275 000 in the manual runs). Writes are asynchronous, so the client does not wait for them. The extra cost comes from the validator, which has to send one write message per entry in the write set, and from the handler. This searches a longer write set on every read.

The success rate was similar for all clients. In each run the per-client rates differed by at most 0.7 percentage points (with 20 writes). With 0 writes all clients committed every transaction.

2.5 Ratio of read and write operations

Experiment. *Provide figures with experimental results comparing the number of transactions and the success rate when executing with different ratio of read and write operations for a fixed amount of operations per transaction (including special cases having only read or write operations).*

Question. *Discuss the impact of the ratio of read and write operations on the success rate, and indicate whether the success rate is the same for the different clients.*

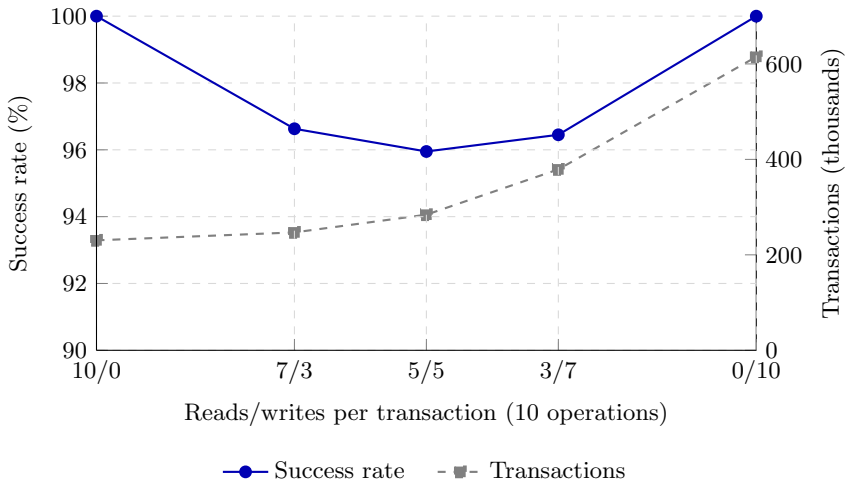


Figure 5: Success rate and number of transactions vs. ratio of read and write operations (average of 10 runs). The success-rate axis starts at 90%.

Reads/Writes	Transactions	Committed	Success rate (%)
10/0	250 127	250 127	100.00
7/3	306 199	295 886	96.63
5/5	365 716	350 960	95.97
3/7	457 263	441 069	96.46
0/10	818 046	818 046	100.00

Table 10: Manual runs with the read/write ratio of 10/0, 7/3, 5/5, 3/7 and 0/10.

Reads/Writes	Transactions	Committed	Success rate (%)
10/0	230 391	230 391	100.00
7/3	247 037	238 712	96.63
5/5	284 128	272 625	95.95
3/7	379 049	365 586	96.45
0/10	614 925	614 925	100.00

Table 11: Average of 10 runs with the read/write ratio of 10/0, 7/3, 5/5, 3/7 and 0/10.

Tables 9 and 10 show that with 10 operations per transaction, both extreme cases have a success rate of 100%. With only reads (10/0), no entries are ever modified, so validation always succeeds. With only writes (0/10), the read set is empty, so there is nothing to validate. A conflict needs both reads and writes, so the success rate is lowest when we have both. The lowest rate is at 5/5 (95.97%), and 7/3 and 3/7 give almost the same rate (about 96.5%). This fits with the number of possible conflicts being roughly proportional to reads $\times$ writes ($7 \cdot 3 = 3 \cdot 7 = 21 < 5 \cdot 5 = 25$).

The number of transactions increases a lot as reads are replaced by writes. It goes from about 250 000 with 10/0 to about 820 000 with 0/10. As explained in Section 2.3, this is because reads are synchronous while writes are not.

At 10/0 and 0/10 all clients committed every transaction. For the mixed ratios the per-client rates differed by at most 0.3 percentage points in each run, so the success rate was the same for all clients.

2.6 Percentage of accessed entries

Code. *Discuss the changes needed in the client so that each one accesses a randomly generated subset of the store: the number of entries in each subset should be the same for all the clients, but the subsets should be different per client and should not contain necessarily contiguous entries.*

Experiment. *Provide figures with experimental results comparing the number of transactions and the success rate when executing with different percentage of accessed entries with respect to the total number of entries.*

Question. *Discuss the impact of the percentage on accessed entries on the success rate, and indicate whether the success rate is the same for the different clients.*

To let each client access its own random subset of the store, `opty:start/6` takes a new parameter `Subset`, the number of entries each client may access (`opty:start/5` uses the whole store). For every client it creates, `opty` generates a random subset of the store using the code from the assignment and passes it to the client instead of the number of entries:

```
subset(Entries, S) ->
    lists:sublist([X || {_, X} <- lists:sort(
        [{rand:uniform(), E} || E <- lists:seq(1, Entries)])], S).
```

Since new random numbers are drawn for every client, the subsets differ per client and is not contiguous. The client stores its subset as a tuple, and in `do_read` and `do_write` the entry is chosen as `element(rand:uniform(tuple_size(Entries)), Entries)` instead of `rand:uniform(Entries)`.

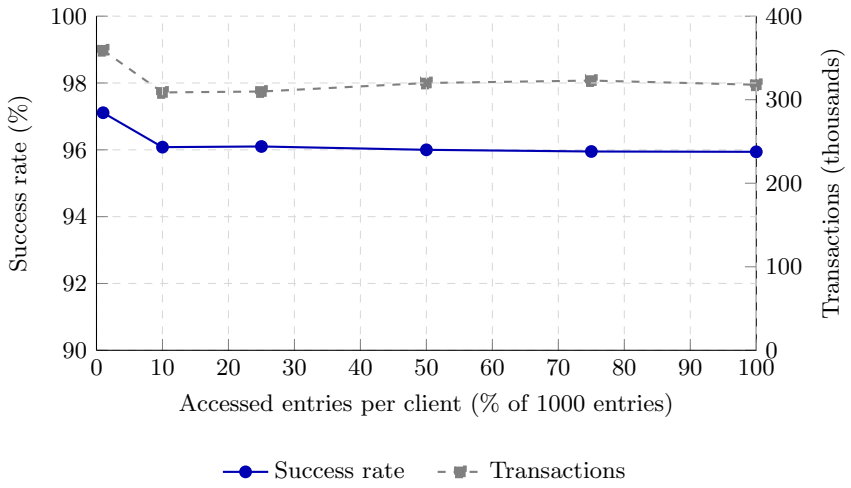


Figure 6: Success rate and number of transactions vs. percentage of accessed entries per client (average of 10 runs). The success-rate axis starts at 90%.

Entries per client	Transactions	Committed	Success rate (%)
10	405 230	398 140	98.25
100	357 200	342 342	95.84
250	367 101	352 851	96.12
500	369 443	354 676	96.00
750	370 888	356 061	96.00
1000	366 188	351 327	95.94

Table 12: Manual runs with entries 10, 100, 250, 500, 750, 1000 accessed per client. Configurations run more than once show the mean of the manual runs.

Entries per client	Transactions	Committed	Success rate (%)
10	359 228	348 763	97.11
100	308 689	296 586	96.08
250	309 841	297 764	96.10
500	320 001	307 204	96.00
750	322 901	309 834	95.95
1000	317 908	305 016	95.94

Table 13: Average of 10 runs with entries 10, 100, 250, 500, 750 and 1000 accessed per client.

Tables 11 and 12 show that the success rate stays at around 96% from 10% to 100% of accessed entries. With only 1% (10 entries per client) being a little higher (97–98%). At first it may look like the smaller subsets should give more conflicts, because each client uses fewer entries more often. However, smaller subsets also overlap less. A given entry that one client reads belongs to another client's subset with probability S/N, and each write of that client hits it with

probability $1/S$. The probability that a write hits the entry is therefore $1/N$, independent of the subset size S. This is why the success rate is almost constant.

With 10 entries per client, a read often finds the entry already in the transaction's own write set, or reads the same entry twice. Fewer distinct entries end up in the read set, so there are fewer entries that can cause a conflict. This also explains the higher number of transactions with 1%, since those reads are served locally by the handler.

Unlike the previous experiments, the success rate is actually different between clients when the subsets are small. Two random subsets of 10 entries out of 1000 overlap in only $10 \cdot 10/1000 = 0.1$ entries on average. This means most clients share no entries with each others. In most of our manual runs with 10 entries, all four clients committed 100% of their transactions. In one run, however, two clients committed only about 86% while the other two committed 100%, which means only those two subsets overlapped. Over the 10 script runs with 10 entries, the per-client rates ranged from 79.1% to 100%, and the gap between the best and worst client in a run was up to 20.9 percentage points. With 100 entries (10%) the gap was at most 2 percentage points, and from 250 entries upwards it was below 1 percentage point, so the clients behave almost the same once the subsets are large enough to overlap with each other.

2.7 Distributed execution

Code. *Discuss the changes needed in the* `opty` *module to ensure that the clients can connect correctly to the server, which must be created in a different remote Erlang instance (named* `opty-srv`*).*

To run the server in a separate Erlang instance we start two nodes on the same machine, `opty-srv` for the server and `opty-cli` for the clients, both with the same cookie (`opty123`). Most of the changes are in `opty:start`.

First, the client node needs to know the name of the server node. At the beginning we hardcoded it, but the hostname of our laptop changed depending on the network, so now `start/5` takes the host from the client's own node name and builds `opty-srv@Host`. We also added `start/6`, where the server node can be passed directly if it is on another machine.

Then `start` sets the cookie and does a `net_adm:ping` to the server node to connect to it. If it gets `pang` it just exits with an error. After that, the server is created on the remote node with `rpc:call(ServerNode, opty, start_server, [Entries])`. This function runs on `opty-srv` and registers the server there as `s`. Since registered names are only local to a node, the clients cannot use `s` directly anymore, so we pass them `{s, ServerNode}`. The clients are only created after `start_server` returns, so the server is already up when they send their first `open`.

We had one problem during testing: if a previous run crashed, the old server was still registered, and the new run reused it even with a different number of entries. Now `start_server` first stops the old `s` (and waits for it to die with a monitor) before creating a new one.

The `stop` function stops the clients, waits for all of them, and then sends `stop` to the remote server, which also stops the validator and the entries. Finally, we fixed a small bug in `client`: if a client did 0 transactions, `100*Ok/Total` crashed with a division by zero, the client never sent `done`, and `stop` got stuck.

To run it:

```
erl -sname opty-srv -setcookie opty123     % terminal 1
erl -sname opty-cli -setcookie opty123     % terminal 2
opty:start(4, 100, 3, 3, 30).              % in opty-cli
```

Experiment. *Record a video providing empirical evidence of the distributed execution working properly.*

In the video LINK we have two terminals, one for each node, and we launch `opty:start(4, 100, 3, 3, 30)` from `opty-cli`. While it is running, `nodes()` in `opty-srv` shows `['opty-cli@Inderjots-MacBook-Air']` and `whereis(s)` gives a pid (`<0.96.0>`), so the two nodes are connected and the server is running on `opty-srv`. After 30 seconds the 4 clients print their results, each one with around 180,000 transactions and a success rate of about 90% (between 89.89% and 89.98%). When we call `whereis(s)` again it returns `undefined`, so the server was stopped from the client node as expected.

Question. *If we run Opty in a distributed Erlang network, indicate in which Erlang instance the client, server, validator, and handler processes are running.*

The clients run in `opty-cli`, since they are spawned there by `opty:start`. The handlers also run in `opty-cli`: the server does not create them, each client creates its own handler with `handler:start` when it gets the `transaction` message, so it is spawned (and linked) on the client's node.

The server runs in `opty-srv`, and so do the validator and all the entries, because they are created by the server itself in `init`. The handler only gets the tuple with the pids of the entries, so every read and the commit go over the network to `opty-srv`, while the checks and writes done by the validator stay local to that node.

2.8 Other concurrency control techniques

Answer the corresponding subsection according to the chosen activity.

2.8.1 Forward validation

Code. *Discuss the changes needed in the code to implement forward validation instead of backward validation.*

Backward validation look at the past: it aborts a transaction if a transaction that already committed changed data it read. Forward validation looks ahead: it aborts a transaction if data it is about to write is still being read by transactions that are running now.

To implement forward validation we only changed `entry`, `handler` and `validator`; `store`, `server`, `client` and `opty` are the same as in the basic (single-node) version.

The main change is in the entry. Instead of a timestamp, each entry keeps a list of its active readers. A transaction is added to the list when it reads the entry, and removed with a `release` message when it commits or aborts, otherwise it would keep blocking writers forever.

The handler becomes simpler. Since reads are not validated anymore, it only remembers which entries it read so they can be released at the end, and it sends its own pid with the commit so the entries can tell the committing transaction apart from the other readers. The validator now sends the checks to the entries

in the write set instead of the read set, and aborts the transaction if any of them has another active reader.

We also tested a version without locking, and our tests found lost updates and transactions that saw an inconsistent state. The problem is that a reader can get in between the check and the write and see some of the new values and some of the old ones. To avoid this, an entry that receives a check stays locked until the validator sends either the new value (`write`) or an `unlock`, and reads wait until then. Because of this, the validator has to wait for all the replies even after it sees an abort, since every checked entry needs to be unlocked.

Experiment. *Provide figures with experimental results on the number of transactions and the success rate to compare forward validation with backward validation for different number of concurrent clients in the system.*

Setup: These experiments were run with a separate script on an Apple M1 Pro (8 cores): each configuration was run 3 times for 4 seconds, alternating between the two implementations, and the figures show the mean and standard deviation. The configurations differ from those in Sections 2.1–2.6, so the numbers are not directly comparable.

		4 clients		16 clients		64 clients	
Scenario	E / R / W	BV	FV	BV	FV	BV	FV
base	100 / 2 / 2	93.2	94.6	69.4	59.9	39.1	9.0
lowcont	1000 / 2 / 2	99.3	99.5	95.2	95.1	81.9	78.5
highcont	10 / 2 / 2	64.9	60.0	29.3	1.5	11.8	0.0
readheavy	100 / 3 / 1	94.8	95.9	74.8	69.7	44.6	16.7
writeheavy	100 / 1 / 3	94.5	95.4	74.1	66.8	44.6	16.0

Table 14: Success rate (%) of backward (BV) and forward (FV) validation. E = entries, R/W = reads/writes per transaction. Read-only and write-only workloads commit 100% in both cases and are omitted.

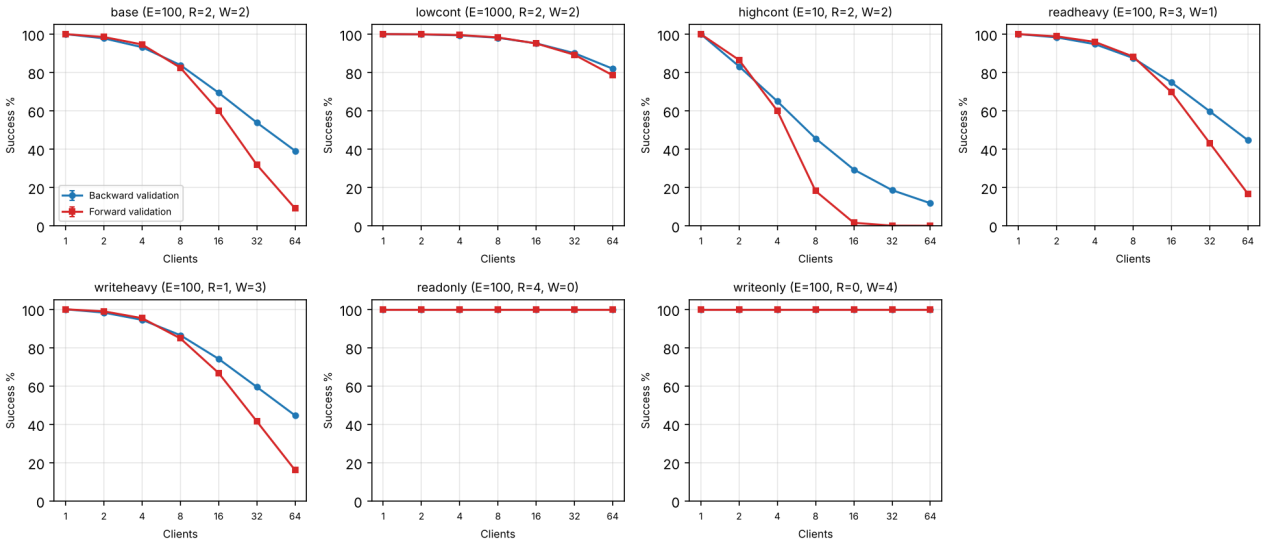


Figure 7: Success rate against the number of concurrent clients for backward (blue) and forward (red) validation, for each workload. E = entries, R/W = reads/writes per transaction.

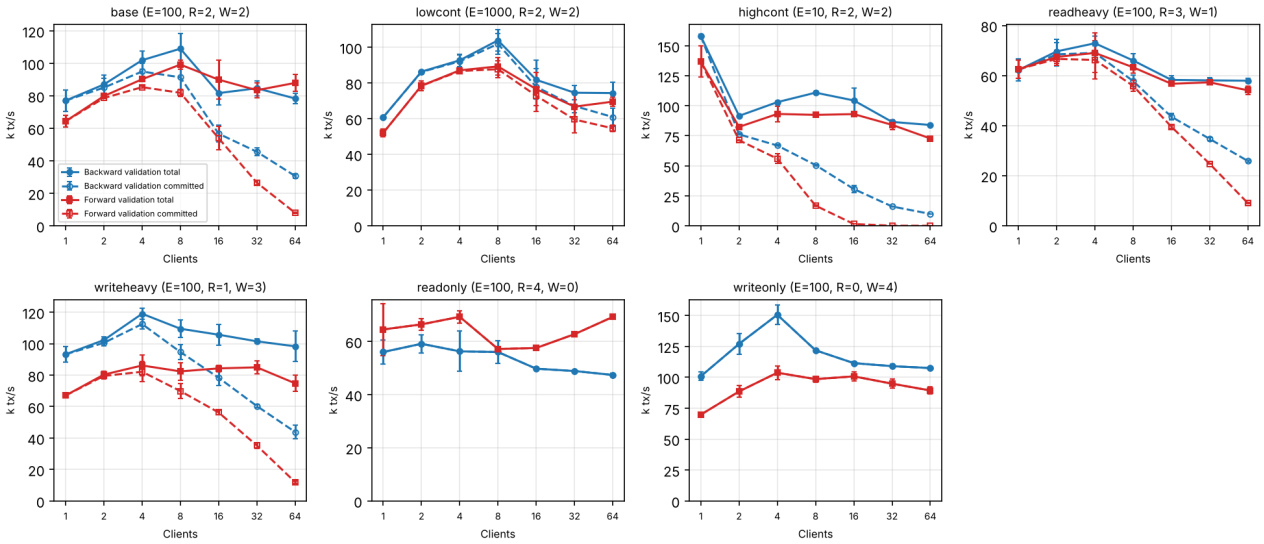


Figure 8: Transactions per second against the number of concurrent clients. Solid lines count all transactions; dashed lines count only committed ones.

Both approaches perform about the same with few clients, but forward validation drops much faster as clients increase, and with 10 entries it practically stops committing (Figure 7). Both systems process a comparable number of transactions (forward validation is somewhat slower when transactions write

and faster when they only read), but under forward validation a much larger share of them abort (Figure 8).

3 Personal opinion

Provide your personal opinion of the seminar, indicating whether it should be included in next year's course or not.

This seminar was a good and educational lab. Completing the missing parts of the code was a good way to understand how Opty works, since we had to understand how the handler, the validator and the entries interact before the system could run. The experiments with different parameters (Sections 2.1–2.6) were interesting, because they showed how optimistic concurrency control behaves under different amounts of contention. However, they felt a bit repetitive, since most of them follow the same pattern and lead to similar conclusions. Overall, we think the seminar should be included in next year's course.